

Optimizing Multi-Depot Vehicle Routing Under Dynamic Demand Conditions using a Learnheuristic Approach

Wenwen Chen
CIGIP

Universitat Politècnica de València
Alcoy, Spain
cwenwen@upv.edu.es

Xabier A. Martin
CIGIP

Universitat Politècnica de València
Alcoy, Spain
xamarsol@upv.es

Raquel Soriano-Gonzalez
CIGIP

Universitat Politècnica de València
Alcoy, Spain
rsorgon@epsa.upv.es

Angel A. Juan
CIGIP

Universitat Politècnica de València
Alcoy, Spain
ajuanp@upv.es

Noelia Fuster
CIGIP

Universitat Politècnica de València
Alcoy, Spain
nfuscom@epsa.upv.es

Abstract—In industries where production centers depend on a steady supply of materials, planning logistics can become complex due to shifting demand patterns. Factors like seasonal changes, regional events, and varying consumer trends contribute to dynamic demand, making it challenging to maintain efficient routes for decision-makers. This paper addresses the Multi-Depot Vehicle Routing Problem (MDVRP) under these dynamic demand conditions, and it presents a learnheuristic methodology that combines a metaheuristic algorithm with a supervised machine learning model. The machine learning model estimates the demand fluctuations and provides this information into the solution construction process rather than relying solely on static demands. The computational experiments demonstrate the learnheuristic approach provides near-optimal solutions that account for demand fluctuations, optimizing routes across multiple depots and minimizing stockouts. This offers an interesting alternative for decision-makers in practical applications to streamline logistics planning and reduce operational costs.

Index Terms—multi-depot vehicle routing problem, optimization, learnheuristics.

I. INTRODUCTION

In industries where production centers depend on a steady supply of materials, planning logistics can become increasingly complex as demand patterns shift over time. The well-known capacitated vehicle routing problem (CVRP) has long been used to model such distribution challenges. In its classical form, the CVRP seeks an optimal set of routes for a fleet of homogeneous vehicles that start and end at a central depot while satisfying the demands of a set of customers. The multi-depot vehicle routing problem (MDVRP) extends this model by considering multiple depots, which introduces the additional complexity of assigning customers to depots and solving several interconnected routing problems. This extension is

particularly relevant in real-life scenarios, where businesses possess several distribution depots and the number of available vehicles per depot is limited. However, recent advances in e-commerce and the emergence of new technologies have further complicated logistics planning. Seasonal variations, regional events, and varying consumer trends are just a few factors that contribute to the dynamic nature of demand. In such an environment, traditional routing methods that focus solely on minimizing distance may not be sufficient to prevent stockouts and reduce operational costs.

This paper presents a learnheuristic approach for the MDVRP under dynamic demand conditions that change based on factors such as seasonal changes, regional events, and business promotions. Learnheuristics combines a metaheuristic algorithm with a supervised machine learning (ML) model [1]. This combination is designed to predict the effects of dynamic demand fluctuations on routing decisions, ensuring that alternative high-quality solutions are available. The proposed learnheuristic algorithm extends a biased-randomized iterated local search (BR-ILS) algorithm proposed in [2] with a trained ML model to predict the demands associated with routing decisions. Figure 1 shows an illustrative example of the MDVRP with dynamic demands. The vehicles depart from the different central depots and serve the customers. The customers' demand can fluctuate based on factors such as weather, business promotions, and holidays. Therefore, the algorithm generates a routing plan that adapts to these estimates, ensuring an efficient distribution of resources.

The contributions of this paper can be summarized as follows: (i) it extends the classical MDVRP to incorporate dynamic demands based on seasonal changes, regional events, and business promotions; and (ii) it proposes a learnheuristic approach that extends a BR-ILS algorithm with a supervised ML model to predict demand fluctuations and integrate them

This work has been partially funded by the Spanish Ministry of Science and Innovation (PID2022-138860NB-I00 and RED2022-134703-T).

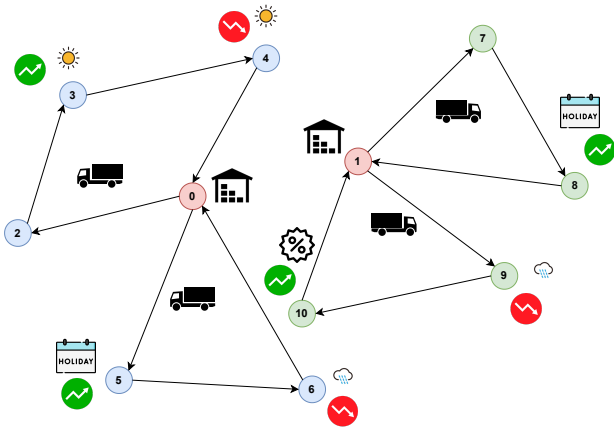


Fig. 1. Schema of the MDVRP with dynamic demands.

into the solution construction process. To the best of our knowledge, this is the first time a learnheuristic algorithm has been proposed to address the MDVRP with dynamic demands that change based on seasonal changes, regional events, and business promotions. This approach offers a novel perspective on how ML techniques can improve decision-making in logistics.

The remainder of this paper is organized as follows. Section II reviews the relevant literature on MDVRP and discusses previous approaches to handling dynamic factors in routing problems. Section III presents the details of the learnheuristic algorithm, including the combination of the BR-ILS metaheuristic with the ML model. Section IV describes the computational experiments, while Section V analyzes the results obtained. Finally, Section VI provides the main conclusions and outlines the directions for future research.

II. RELATED WORK

The MDVRP has been extensively studied in the scientific literature due to its broad applicability in logistics and supply chain management, especially in industries with complex distribution networks. The solutions approached proposed to address the MDVRP have evolved from classical heuristics to metaheuristics, and more recently, to ML-based approaches.

Initial solution approaches for the MDVRP adapted techniques from the single-depot CVRP. [3] extended the Clarke and Wright savings heuristic to handle multiple depots [4]. [5] developed a two-stage heuristic that first assigned customers to depots and then determined the routes for each depot. Subsequently, [6] introduced a multi-terminal sweep method, which assigns customers to depots based on angular coordinates and then sequences deliveries. [7] enhanced savings-based heuristics by incorporating additional criteria for route selection. [8] proposed a branch-and-bound approach to find exact solutions, but its high computational cost limited scalability.

While heuristic methods provided promising initial solutions to the MDVRP, they often fell short of optimality, especially as problem complexity increased. Exact methods, although capable of finding optimal solutions, become computationally

infeasible for large instances due to the NP-hard nature of the problem. Consequently, metaheuristics emerged as efficient alternatives, offering more effective strategies for navigating large and complex search spaces. [9] introduced a record-to-record improvement heuristic, which allows controlled deterioration during the search to escape local optima. [10] and [11] applied a tabu search (TS) metaheuristic to avoid cycling back to previously visited solutions, significantly improving solution quality. Similarly, [12] proposed a genetic algorithm (GA) with clustering-based strategies to enhance GA performance. [13] proposed a hybrid GA that combined classical heuristics to refine solution quality. A significant advancement was made by [14], who developed an adaptive large neighborhood search (ALNS) metaheuristic. This solution approach systematically destroyed and repaired parts of the solution to explore a broader search space, improving the ability to solve large-scale instances and becoming one of the most effective metaheuristics for the MDVRP. Recent studies have introduced innovative approaches to enhance solution quality for the MDVRP. [15] introduced a hybrid approach that combines a TS metaheuristic with differential evolution techniques to optimize routing efficiency in complex MDVRP scenarios. Similarly, [2] combined an iterated local search (ILS) metaheuristic with biased-randomization techniques to provide efficient solutions for the problem. [16] proposed a stochastic memetic algorithm with interdependent depot operations to manage the complexities arising from depot interdependencies effectively, improving overall routing performance. Recently, [17] developed a hybrid ALNS method that integrates depot location decisions, allowing for simultaneous optimization of depot placements and vehicle routes.

Advancements in ML techniques have significantly enhanced optimization strategies for the MDVRP, leading to more adaptive and efficient solutions in real-world scenarios with uncertainty. [18] combined statistical learning models with metaheuristics, optimizing customer allocation based on historical demand patterns. This approach enabled dynamic adjustments to customer assignments, improving routing efficiency in response to changing demand. [19] explored ML-based heuristic operator selection during the optimization process, adapting to various problem instances and improving dynamic optimization efficiency. Deep learning has also made a significant impact on MDVRP research. [20] introduced DeepMDV, a neural network-based model designed to optimize routes and vehicle assignments. DeepMDV employs deep learning techniques to learn complex patterns in routing data, enabling it to generate high-quality solutions for large-scale instances. Moreover, reinforcement learning has emerged as a promising strategy for MDVRP optimization. [21] applied reinforcement learning to optimize routes in dynamic demand environments. Their approach allows the system to learn optimal routing policies through interactions with the environment, effectively adapting to real-time changes in demand. [22] developed a multi-agent reinforcement learning model to improve vehicle coordination and real-time operational efficiency. This model employs multiple agents that collaboratively learn

optimal strategies for vehicle routing, enhancing coordination and responsiveness to dynamic conditions.

Figure 2 illustrates the continuous evolution of the solution approaches proposed to solve the MDVRP in the scientific literature. Since the MDVRP has been a subject of research for decades, this representation highlights its development over time and demonstrates how it remains an area of ongoing study and transformation.

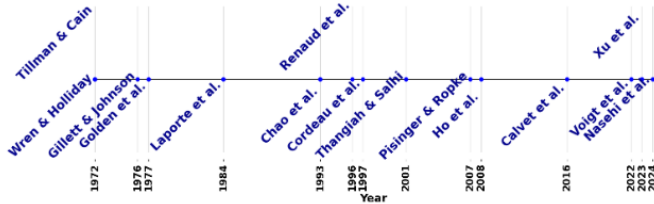


Fig. 2. Evolution of the MDVRP literature over time.

Despite the advancements in solution approaches to address the MDVRP, optimization approaches under dynamic conditions remain a challenge in the scientific literature. Specifically, ML-based methods adapt well to dynamic environments by quickly adjusting to new data, they often lack an explicit optimization step, whereas traditional optimization approaches provide high-quality solutions but struggle to handle real-time changes effectively. This limits the adoption of these solution approaches in real-world scenarios with uncertainty and dynamic factors. Therefore, hybrid approaches that combine both techniques are an interesting area for further research to achieve real-time solutions in logistics.

III. SOLUTION APPROACH

This study proposes a learnheuristic approach to solve MDVRP under dynamic demand conditions. The solution approach combines a BR-ILS metaheuristic algorithm with a supervised ML model to generate solutions that estimate the demand fluctuations associated with routing decisions. This combination enables the optimization process to better adapt to dynamic demand conditions. The BR-ILS algorithm provides near-optimal solutions for the MDVRP, and the ML model is trained on previous historical demand data.

Algorithm 1 details the learnheuristic algorithm, which extends the BR-ILS proposed by [2] by integrating demand prediction in the solution construction process. First, the algorithm computes a priority list of potential customers for each depot. Therefore, for each depot, the priority list contains all customers sorted in decreasing order according to a marginal savings criterion. The marginal savings criterion measures the cost difference between assigning a customer to a specific depot versus the next best alternative, prioritizing customers with the highest savings for their closest depot. Subsequently, the learnheuristic generates an initial customer-to-depot assignment map using these priority lists. In particular, the algorithm iteratively selects a depot following different alternative criteria (i.e., random, round-robin, most remaining capacity) and assigns a node in a biased-randomized

manner from its priority list. The selection process follows a geometric distribution followed by a parameter $\beta \in (0, 1)$. This biased-randomization method allows the generation of different assignment maps. Moreover, the learnheuristic employs the estimated demands of the customers, provided by the pretrained ML model, to generate more accurate allocation maps that account for the real demand of the customers. Once the initial allocation map is generated, the algorithm generates an initial solution by solving each nodes-depot pair using the Clarke and Wright savings (CWS) heuristic [4]. This initial solution is assigned as the base and best solution, and it is inserted into a pool of “elite” solutions for further refinement. Next, the ILS metaheuristic iteratively explores the solution space until a stopping criterion is met (i.e., a maximum number of iterations). At each iteration, the ILS metaheuristic perturbs the assignment map associated with the base solution, which reallocates a percentage of nodes to different depots according to the aforementioned criteria. Subsequently, the algorithm generates a new solution with this new allocation map using the CWS heuristic. If the new solution violates the depots’ vehicle limit, the perturbation process is repeated. Otherwise, if the new solution is better than the current base solution, the base solution is updated. If the new solution is better than the best-found solution, the solution is improved using a local search process. Moreover, the best-found solution is updated and the solution is inserted into the pool of elite solutions. To diversify the search process, non-improving solutions may be accepted based on a credit mechanism [23]. This approach avoids getting stuck in local optima by occasionally accepting suboptimal solutions, promoting the exploration of a broader solution space. The credit value is adjusted based on the cost difference between the new and base solutions, allowing slight deterioration when it enhances search diversity. Once the maximum number of iterations is reached, the pool of elite solutions is refined as a post optimization process. This refinement employs the SR-GCWS-CS algorithm proposed in [24] to generate more efficient solutions for the assignment maps of the elite solutions. Readers are encouraged to refer to [24] for a detailed explanation of the implementation. Finally, the algorithm return the best elite solution from the procedure.

IV. COMPUTATIONAL EXPERIMENTS

The computational experiments were conducted on a Linux Mint 22 system with an AMD Ryzen Threadripper 7970X at 4 GHz and 128 GB RAM. The learnheuristic algorithm was implemented in Python 3.12. To maintain consistency and facilitate a direct comparison with the base BR-ILS algorithm proposed in [2], the same parameter values were given to our learnheuristic algorithm. Specifically, each instance was run five times with different seeds for the random number generator, and a maximum of 300,000 iterations per execution. The biased-randomization parameter for the geometric distribution related to the allocation problem was chosen in the interval (0.5, 0.8). Similarly, the corresponding parameter for the refinement algorithm was chosen within the interval

Algorithm 1 Learnheuristic algorithm.

```

1: function LH-MDVRP(inputs, maxTime, predModel)
2:   priorityList  $\leftarrow$  GENPRIORITYLIST(inputs)
3:   initMap  $\leftarrow$  GENMAP(inputs, predModel, priorityList)
4:   initSol  $\leftarrow$  GENFEASIBLESOL(inputs, initMap)
5:   baseSol  $\leftarrow$  initSol
6:   bestSol  $\leftarrow$  initSol
7:   eliteSols  $\leftarrow$   $\emptyset$ 
8:   eliteSols  $\leftarrow$  UPDATE(eliteSols, initSol)
9:   i  $\leftarrow$  0
10:  while i < maxIter do
11:    newMap  $\leftarrow$  PERTURBMAP(baseSol, predModel, priorityList)
12:    newSol  $\leftarrow$  GENFEASIBLESOL(inputs, newMap)
13:    if ISFEASIBLE(newSol) then
14:       $\Delta \leftarrow$  COST(newSol) - COST(baseSol)
15:      if  $\Delta < 0$  then
16:        credit  $\leftarrow$  - $\Delta$ 
17:        baseSol  $\leftarrow$  newSol
18:        if COST(newSol) < COST(bestSol) then
19:          newSol  $\leftarrow$  LOCALSEARCH(inputs, newSol)
20:          bestSol  $\leftarrow$  newSol
21:          eliteSols  $\leftarrow$  UPDATE(eliteSols, newSol)
22:        end if
23:      else if  $0 < \Delta \leq \text{credit}$  then
24:        credit  $\leftarrow$  0
25:        baseSol  $\leftarrow$  newSol
26:      end if
27:    end if
28:    i  $\leftarrow$  i + 1
29:  end while
30:  for eliteSol  $\in$  eliteSols do
31:    eliteSol  $\leftarrow$  REFINESOLUTION(inputs, eliteSol, predModel)
32:    if COST(eliteSol) < COST(bestSol) then
33:      bestSol  $\leftarrow$  eliteSol
34:    end if
35:  end for
36:  return bestSol
37: end function

```

(0.1, 0.2). Finally, the degree of perturbation at each iteration of the ILS was selected randomly in the range 10% – 50%.

To assess the effectiveness of the learnheuristic approach and ensure comparability with existing methods, 23 benchmark instances of the MDVRP proposed by [11] were tested. These classical instances consider a limited fleet of vehicles at each depot with homogeneous capacities. Moreover, several instances require a maximum route length constraint, which can be enforced in the CWS heuristic merging process.

For comparison purposes, the computational experiments provide two approaches to solve the dynamic MDVRP: (i) the original BR-ILS algorithm, where the demand at each node remains unchanged throughout the solution construction; and (ii) the proposed learnheuristic approach, where the demand at each node is estimated by the ML model. Figure 3 illustrates the different approaches and the validation process. Notice that the static demand provided by the benchmark instances is employed by the BR-ILS algorithm to generate static solutions that do not account for demand fluctuations. In contrast, the learnheuristic algorithm (BR-ILS LH) incorporates dynamic demand considerations into the optimization process. Specifically, the White Box (WB) model is trained using data generated by the Black Box (BB), which represents real-world demand patterns. The demand estimates provided by the WB are employed by the learnheuristic algorithm described in

Algorithm 1 to generate dynamic solutions that better reflect real-world conditions. Subsequently, both static and dynamic solutions are “executed” in a real-world environment. In other words, the solutions are simulated in a real scenario under dynamic demand conditions that follow the BB. This step outputs the actual cost of both the static solution (OBS-SC) and the dynamic solution (OBS-DC). The remainder of the section will describe the implementation details of the demand patterns formulation, BB model, WB model, and the solution execution procedure.

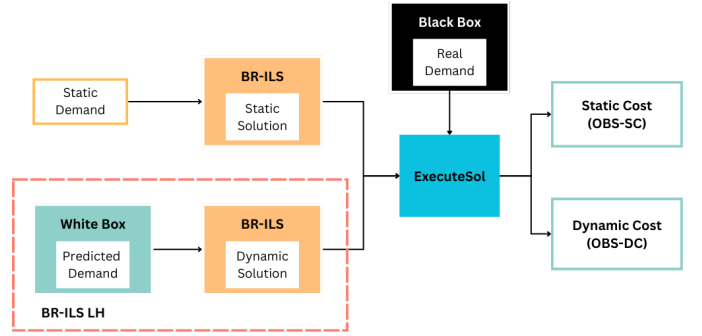


Fig. 3. Schema of the approaches for the computational experiments.

A. Demand Patterns Formulation

The dynamic demand patterns follow a real-world scenario, where seasonal variations, regional events, and varying consumer trends are just a few factors that contribute to the dynamic nature of demand. Specifically, the experiments consider the following factors to influence the demand:

- Seasonal variations: Fluctuations in customer demand over time.
- Holidays and regional events: Special days that alter shopping patterns.
- Promotions: Temporary price discounts or marketing campaigns that affect purchase behavior.

Based on these factors, the BB model applies a set of predefined heuristic rules to generate the actual demands, while the WB uses a ML model trained in random observations provided by the BB model to predict the demand fluctuations.

1) *Black Box Model*: The BB model defines the demand fluctuations using an analytical expression influenced by the aforementioned external factors, such as temperature, promotions, and holidays. Therefore, the demand ratio at each node is determined by the following equation:

$$r = 1 + a \frac{T - \mu_T}{\sigma_T} + b \delta_h + c \mathbb{I}(\delta_p = 1) + d \mathbb{I}(\delta_p = 2) \quad (1)$$

where:

- T represents the temperature, with a mean temperature of $\mu_T = 22$ °C and a standard deviation of $\sigma_T = 8$ °C.
- δ_h indicates whether today is a holiday or not.
- δ_p represents the promotion intensity with 0, 1, and 2 representing none, low, and high, respectively.

- $\mathbb{I}$ denotes an indicator function. If $\delta_p = 0$, both contributions of c and d vanish, if $\delta_p = 1$ only c is active, and if $\delta_p = 2$ only d is active.

Moreover, external factors will influence the customers in a different manner depending on their type. Specifically, we define four types of customers as follows:

- Type 0: Customer demand is unchanged.
- Type 1: Customer demand is influenced by temperature.
- Type 2: Customer demand is influenced by the holiday factor.
- Type 3: Customer demand is influenced by the promotion levels.

For the experiments, the parameters a , b , c , and d were assigned values of 0.2, 0.1, 0.05, and 0.1, respectively. These parameters are assigned their specified values only when the corresponding external factor is active for a given customer; otherwise, they are set to zero. Finally, the actual demand at a given node i is computed as follows:

$$D_i^* = r \cdot d_i \quad (2)$$

where r is the demand ratio computed by the BB and d_i is the base demand of the node.

2) *White Box Model*: The WB model is trained using data generated by the BB model. To generate this data, we generate 500 samples with random node types, temperatures ranging from 0 to 35 °C, holiday indicators (0 or 1), and promotion levels (0, 1, or 2), following realistic distributions. This process ensures diversity in demand fluctuations and allows the ML model to learn from different dynamic conditions. The WB model is implemented using an XGBoost regression model and configured with the following parameters:

- Objective function: Squared error regression
- Number of estimators: 100
- Learning rate: 0.1
- Maximum depth: 5

The dataset is split into 70% for training and 30% for validation purposes. Once the model is trained, we estimate the demand ratio $\hat{r}$ of each sample in the test set, which is then used to compute the predictive demand:

$$\hat{D}_i = \hat{r} \cdot d_i \quad (3)$$

To evaluate the effectiveness of the WB model, we compare its predicted demand against the actual demand adjustments from the BB model using mean squared error (MSE). This assessment ensures that the WB accurately approximates the demand variations defined by the BB. Once the validation process is finished, the WB model is ready to predict the demand ratio of the customers with different external factors. Specifically, the WB model is serialized to disk and de-serialized when the learnheuristic algorithm needs to estimate the demand fluctuations in the solution construction process.

B. Solution Execution

Algorithm 2 summarizes the solution execution procedure. Each time both the static and dynamic solutions are executed in a real-world scenario, different external factors are generated randomly to ensure robustness in the results. In other words, the solutions are executed with different external factors to demonstrate their effectiveness in a dynamic scenario. First, the execution procedure initializes the total cost of the solution and iterates through all sub-solutions (i.e., all routes that belong to a particular depot). For each sub-solution, the depot is retrieved and the routes for that sub-solution are iterated. For each route, the procedure computes the actual demand for all the customers in the route following the formula from the BB. If total demand exceeds the vehicle capacity, an additional replenishment cost is incurred. The replenishment mechanism simulates the real-world constraint where vehicles must return to the depot to reload once they reach their capacity limit, generating additional travel costs. This penalization cost ensures a fair evaluation of the effectiveness of demand prediction for the two static and dynamic approaches.

Algorithm 2 Solution execution in a real-world scenario.

```

1: function EXECUTESOL(sol, inputs)
2:   cost  $\leftarrow$  0
3:   for subSol  $\in$  sol.subSolutions do
4:     depot  $\leftarrow$  subSol.depot
5:     subCost  $\leftarrow$  0
6:     for route  $\in$  subSol.routes do
7:       actualDemand  $\leftarrow$  CALCULATEDEMAND(route)
8:       extCost  $\leftarrow$  0
9:       if actualDemand > inputs.vCap then
10:        load  $\leftarrow$  inputs.vCap
11:        routeCost  $\leftarrow$  route.cost
12:        for edge  $\in$  route.edges do
13:          node  $\leftarrow$  edge.end
14:          remaining  $\leftarrow$  node.actualDemand
15:          while remaining > 0 do
16:            if load  $\geq$  remaining then
17:              load  $\leftarrow$  load - remaining
18:              remaining  $\leftarrow$  0
19:            else
20:              remaining  $\leftarrow$  remaining - load
21:              load  $\leftarrow$  inputs.vCap
22:              extCost  $\leftarrow$  extCost + dist(node, depot)
23:              extCost  $\leftarrow$  extCost + dist(depot, node)
24:            end if
25:          end while
26:        end for
27:        routeCost  $\leftarrow$  routeCost + extCost
28:      end if
29:      subCost  $\leftarrow$  subCost + routeCost
30:    end for
31:    cost  $\leftarrow$  cost + subCost
32:  end for
33:  sol.cost  $\leftarrow$  cost
34:  return sol
35: end function

```

V. RESULTS AND DISCUSSION

Table I presents a summary of the computational results obtained for the different scenarios. The first section of the table displays the instance name, number of customers (n),

number of available vehicles (m), number of depots (d), maximum route length if applicable, and maximum vehicle capacity (Q). The subsequent section of the table reports the obtained results for the static scenario where demands are static and known in advance. The first column shows the best-known solutions (BKS) as reported by [11]. The next column reports the best-found solutions (OBS) reported by the BR-ILS algorithm provided in [2]. The last column depicts the percentage gaps (GAP (%)) of the OBS with respect to the BKS. Note that the average percentage gap of our solution approach is 0.62, which underscores the quality of the base BR-ILS algorithm. The last section of the table reports the obtained results for the dynamic scenario where demands fluctuate based on dynamic factors. The first column shows the actual cost of the OBS (OBS-SC) when subject to fluctuating demands. Specifically, the best-found solution by the BR-ILS algorithm is executed in an environment with dynamic demands, and the actual cost is obtained. The last column reports the actual cost of the solutions (OBS-DC) provided by the learnheuristic algorithm. This approach accounts for the demand fluctuations during the solution construction process.

TABLE I
COMPUTATIONAL RESULTS FOR THE DIFFERENT SCENARIOS.

Instances						Static Scenario			Dynamic Scenario	
Instance	n	m	d	Max Route Length	Q	BKS	OBS	GAP (%)	OBS-SC	OBS-DC
p01	50	4	4	N/A	80	576.87	578.20	0.23	598.30	586.10
p02	50	2	4	N/A	160	473.53	476.66	0.66	519.26	480.04
p03	75	3	5	N/A	140	641.19	641.19	0.00	641.19	639.31
p04	100	8	2	N/A	100	1001.59	1023.08	2.15	1118.31	1059.04
p05	100	5	2	N/A	200	750.03	758.17	1.09	789.79	752.03
p06	100	6	3	N/A	100	876.50	880.52	0.46	979.36	875.45
p07	100	4	4	N/A	100	885.80	891.63	0.66	986.17	919.97
p08	249	14	2	310	500	4437.68	4443.75	0.14	5078.46	4454.57
p09	249	12	3	310	500	3900.22	3913.93	0.35	3913.93	3840.37
p10	249	8	4	310	500	3663.02	3667.36	0.12	3850.87	3622.99
p11	249	6	5	310	500	3554.18	3556.97	0.08	3980.43	3601.58
p12	80	5	2	N/A	60	1318.95	1318.95	0.00	1318.95	1313.35
p13	80	5	2	200	60	1318.95	1318.95	0.00	1318.95	1318.95
p14	80	5	2	180	60	1360.12	1360.12	0.00	1360.12	1360.12
p15	160	5	4	N/A	60	2505.42	2539.76	1.37	2639.76	2571.36
p16	160	5	4	200	60	2572.23	2585.37	0.51	2585.37	2585.37
p17	160	5	4	180	60	2709.09	2720.23	0.41	2720.23	2731.37
p18	240	5	6	N/A	60	3702.85	3718.77	0.43	3803.62	3784.95
p19	240	5	6	200	60	3827.06	3855.77	0.75	3855.77	3861.24
p20	240	5	6	180	60	4058.07	4063.64	0.14	4063.64	4085.92
p21	360	5	9	N/A	60	5474.84	5633.40	2.90	5689.97	5670.83
p22	360	5	9	200	60	5702.16	5795.47	1.64	5852.04	5786.29
p23	360	5	9	180	60	6095.46	6112.17	0.27	6112.17	6112.17
Average						2669.82	2689.31	0.62	2772.90	2696.23

Figure 4 provides an overview of Table I about the performance of our learnheuristic approach for the dynamic scenario. The horizontal axis represents the actual costs obtained by the BR-ILS and learnheuristic algorithms, while the vertical axis shows the percentage gaps with respect to the BKS. The obtained results show an average percentage gap of 4.44 for the OBS-SC, while the OBS-DC presents an average percentage gap of 1.05. These results confirm the effectiveness of the learnheuristic approach, as the static solutions provided by the BR-ILS algorithm fail to adapt effectively to the dynamic demand fluctuations. Therefore, incorporating estimations of the dynamic factors into the optimization process leads to significantly superior results compared to solving the MDVRP with static demands and subsequently applying the solution in a real-world context with dynamic factors.

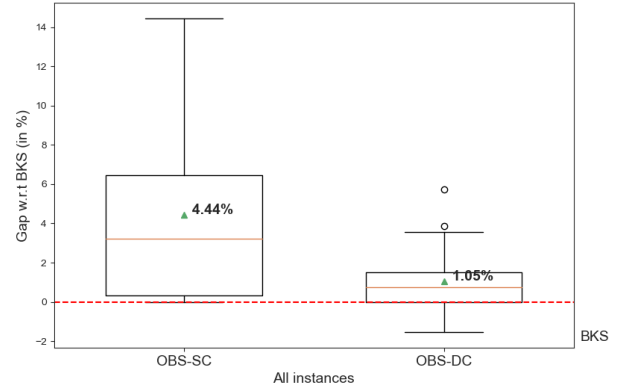


Fig. 4. Percentage gaps of the solution approaches in a dynamic scenario.

To further illustrate the efficiency of the learnheuristic approach, we conducted an analysis using the p06 instance. The instance was chosen randomly from the benchmark dataset. Figure 5 and 6 illustrate the solutions provided by the BR-ILS algorithm and the learnheuristic approach. The depots are represented by red squares, while the routes generated from the different solution approaches are shown as lines of different colors (which differentiates them from one another).

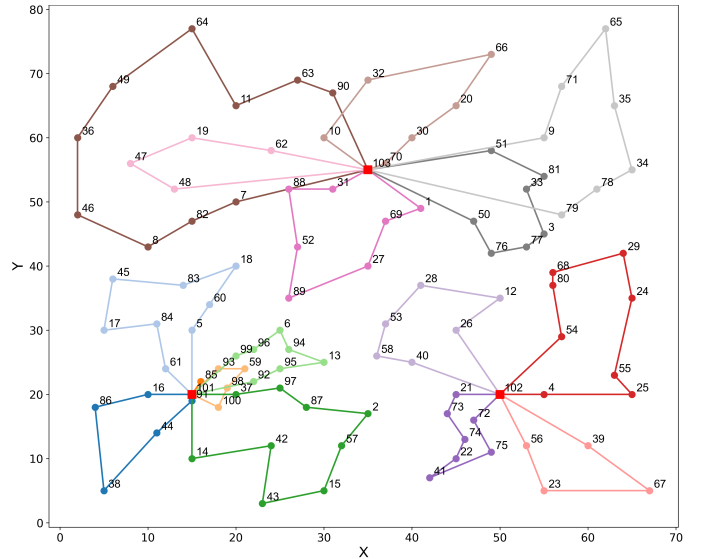


Fig. 5. BR-ILS solution for the p06 instance.

Table II provides a comparative evaluation of the BR-ILS and learnheuristic approaches. For each solution, the table displays the route identifiers, visited customers, estimated demand, real demand satisfied, and demand difference.

Note that the BR-ILS approach generates routes based on static demand without explicitly considering demand fluctuations. Consequently, the average estimated demand per route is 92.41, whereas the actual demand is 89.72, leading to an average deviation of -2.69 units per route. In particular, several

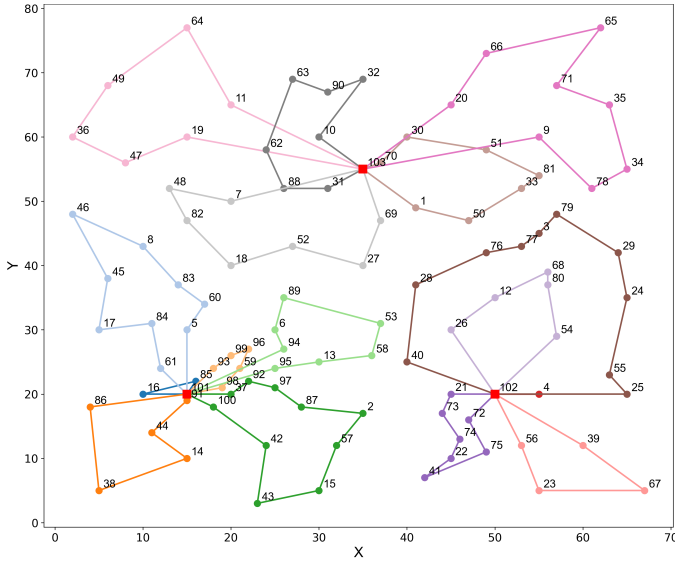


Fig. 6. BR-ILS LH solution for the p06 instance.

TABLE II
COMPARISON OF THE TWO SOLUTIONS FOR THE P06 INSTANCE.

BR-ILS Solution				
Route	Route Path	Estimated Demand	Real Demand	Demand Difference
1	101-91-44-38-86-16-101	89.00	89.00	0.00
2	101-61-84-17-45-83-18-60-5-101	90.00	94.79	4.79
3	101-85-101	41.00	45.10	4.10
4	101-100-98-59-93-101	77.00	84.70	7.70
5	101-37-97-87-2-57-15-43-42-14-101	100.00	91.90	-8.10
6	101-99-96-6-94-13-95-92-101	95.00	68.05	-26.95
7	102-4-25-55-24-29-68-80-54-102	99.00	100.94	1.94
8	102-39-67-23-56-102	91.00	86.44	-4.56
9	102-72-75-41-22-74-73-21-102	94.00	96.63	2.63
10	102-40-58-53-28-12-26-102	93.00	88.31	-4.69
11	103-90-63-11-64-49-36-46-8-82-7-103	100.00	100.05	0.05
12	103-70-30-20-66-32-10-103	99.00	75.92	-23.08
13	103-31-88-52-89-27-69-1-103	92.00	74.88	-17.12
14	103-62-19-47-48-103	99.00	100.28	1.28
15	103-51-81-33-3-77-76-50-103	100.00	92.91	-7.09
16	103-79-78-34-35-65-71-9-103	99.00	91.21	-7.79
Avg	-	92.41	89.72	-2.69
Learnheuristic Solution				
Route	Route Path	Estimated Demand	Real Demand	Demand Difference
1	101-85-16-101	63.50	64.10	0.60
2	101-61-84-17-45-46-8-83-60-5-101	92.90	92.79	-0.11
3	101-91-44-14-38-86-101	90.10	90.00	-0.10
4	101-98-59-96-99-93-101	86.70	86.94	0.24
5	101-100-42-43-15-57-2-87-97-92-37-101	92.70	90.90	-1.80
6	101-95-13-58-53-89-6-94-101	98.70	89.12	-9.58
7	102-4-102	20.70	20.90	0.20
8	102-39-67-23-56-102	86.50	86.44	-0.06
9	102-72-75-41-22-74-73-21-102	96.80	96.63	-0.17
10	102-26-12-68-80-54-102	96.60	96.60	0.00
11	102-40-28-76-77-3-79-29-24-55-25-102	99.70	97.40	-2.30
12	103-70-30-51-81-33-50-1-103	96.70	96.59	-0.11
13	103-20-66-65-71-35-34-78-9-103	97.50	89.63	-7.87
14	103-19-47-36-49-64-11-103	98.10	98.08	-0.02
15	103-31-88-62-63-90-32-10-103	90.30	85.95	-4.35
16	103-69-27-52-18-82-48-7-103	99.10	99.05	-0.05
Avg	-	92.31	91.39	-0.92

routes exhibit significant deviations, such as route 6 (-26.95 units) and route 12 (-23.08 units), indicating underestimations that could result in sub-optimal routes and wasted resources. Moreover, four routes exceed the vehicle capacity limit of 100, requiring additional depot returns to meet the demand. Consequently, the OBS-SC reported is 979.36, which exceeds the OBS cost of 880.52. These discrepancies suggest that static optimization methods struggle to adapt to fluctuating demand effectively, potentially increasing operational inefficiencies. In contrast, the learnheuristic algorithm integrates demand

estimates into the solution construction process. This yields an average estimated demand of 92.31, while the actual demand is 91.39, which leads to a minor deviation of -0.92 units per route. Specifically, demand differences per route remained minimal, with most routes maintaining a ± 1 units range, demonstrating that the predictive model effectively captured the demand patterns. Moreover, no routes in the learnheuristic solution exceed the vehicle capacity limit, ensuring efficient vehicle utilization and no depot returns which increase the solution cost. Therefore, this results in the OBS-DC of 875.45, which is significantly lower than the OBS-SC of 979.36, highlighting the cost-effectiveness of incorporating predictive modeling into the routing decision process.

Finally, we discuss scalability aspects and potential challenges in real-world scenarios. First, the learnheuristic approach generates and refines initial solutions within seconds for instances with up to 360 customers and 9 depots. This suggests the methodology can handle networks of hundreds or even thousands of nodes and dozens of depots. For larger instances, applying customer or depot clustering, warm-start initialization or additional computing resources could help maintain the scalability and computational efficiency of the learnheuristic approach. Compared to the original BR-ILS algorithm, our method delivers similar performance while also adapting solutions to demand fluctuations, providing a clear advantage in dynamic environments. In particular, the additional overhead of combining the pre-trained ML model with the BR-ILS algorithm is minimal, as the model is loaded once at the start of the algorithm execution, and the demand prediction is almost instantaneous. However, the Python implementation of the algorithm, instead of Java as the original one, introduces some runtime performance overhead. This drawback is overlooked as Python's native support for ML libraries makes the integration with the model seamless. When moving to real-world settings, practical constraints—such as depot opening hours, vehicle availability, service-level requirements and contractual delivery windows—must be embedded in the model, and domain experts should guide the parametrization of demand-fluctuation forecasts to capture company-specific patterns and ensure operational feasibility. Overall, the learnheuristic framework offers a robust, scalable solution for dynamic vehicle routing and stands ready for practical deployment in complex logistics networks.

VI. CONCLUSIONS

This paper addressed the MDVRP under dynamic demand conditions and presented a learnheuristic approach that combines a BR-ILS with a supervised ML model. The proposed algorithm incorporates demand predictions directly into the solution construction process, allowing for more adaptive and effective routing decisions. Computational experiments show the importance of incorporating dynamic estimates during the search for solutions, as the average gap for the dynamic approach is consistently lower than for the static approach. Therefore, the learnheuristic framework constitutes a promis-

ing methodology for solving routing problems under dynamic conditions.

Several future research lines are highlighted as follows: (i) to adapt the proposed learnheuristic algorithm to other variants of the MDVRP, adding real-world characteristics and constraints; (ii) to consider more realistic ML models to capture more diverse and realistic interactions between customers and demands; and (iii) to extend the learnheuristic approach to a sim-learnheuristic methodology [25], incorporating other forms of uncertainty, such as stochastic variability in travel distances, through simulation modeling techniques.

MATCH & CONTRIBUTION

This contribution aligns well with the theme of the IEEE 2025 conference on “AI-driven Industrial Transformation: Digital Leadership in Technology, Engineering, Innovation & Entrepreneurship”. The paper presents a novel learnheuristic approach that integrates supervised machine learning with metaheuristics to address the Multi-Depot Vehicle Routing Problem (MDVRP) under dynamic demand conditions, a core challenge in modern industrial logistics. Using AI to estimate demand fluctuations and enhance decision-making in route optimization, this work exemplifies how data-driven engineering and AI methodologies can drive technological innovation and improve operational efficiency. The proposed approach supports digital leadership by enabling industries to adapt logistics planning to real-time data, reducing costs, and enhancing supply chain resilience. This makes it a strong and relevant contribution to the conference’s focus on AI-enabled transformation in engineering and industrial systems.

ACKNOWLEDGMENT

The authors sincerely thank Coca-Cola Europacific Partners for their generous support and collaboration, which have been instrumental in the success of this work.

REFERENCES

- [1] L. Calvet, J. de Armas, D. Masip, and A. A. Juan, “Learnheuristics: hybridizing metaheuristics with machine learning for optimization with dynamic inputs,” *Open Mathematics*, vol. 15, no. 1, pp. 261–280, 2017.
- [2] A. A. Juan, I. Pascual, D. Guimarans, and B. Barrios, “Combining biased randomization with iterated local search for solving the multidepot vehicle routing problem,” *International Transactions in Operational Research*, vol. 22, no. 4, pp. 647–667, 2015.
- [3] F. A. Tillman and J. C. Cain, “An upper bound algorithm for the single and multiple terminal delivery problem,” *Management Science*, vol. 18, no. 11, pp. 664–682, 1972.
- [4] G. Clarke and J. W. Wright, “Scheduling of vehicles from a central depot to a number of delivery points,” *Operations Research*, vol. 12, no. 4, pp. 568–581, 1964.
- [5] A. Wren and A. Holliday, “Computer scheduling of vehicles from one or more depots to a number of delivery points,” *Operational Research Quarterly*, vol. 23, no. 3, pp. 333–344, 1972.
- [6] B. E. Gillett and J. E. Johnson, “Multi-terminal vehicle dispatch algorithm,” *Omega*, vol. 4, no. 6, pp. 711–718, 1976.
- [7] B. L. Golden, A. A. Assad, and R. Dahl, “Analysis of a large scale vehicle routing problem with an improved heuristic,” *Transportation Research*, vol. 11, no. 2, pp. 123–128, 1977.
- [8] G. Laporte, Y. Nobert, and S. Taillefer, “Solving a family of multi-depot vehicle routing and location-routing problems,” *Transportation Science*, vol. 18, no. 1, pp. 40–46, 1984.

- [9] I. Chao, B. Golden, and E. Wasil, “A new heuristic for the multi-depot vehicle routing problem that improves upon deterministic methods,” *Journal of the Operational Research Society*, vol. 44, no. 6, pp. 579–584, 1993.
- [10] J. Renaud, F. F. Boctor, and G. Laporte, “A tabu search heuristic for the multi-depot vehicle routing problem,” *Computers & Operations Research*, vol. 23, no. 3, pp. 229–235, 1996.
- [11] J. F. Cordeau, G. Laporte, and A. Mercier, “A tabu search heuristic for periodic and multi-depot vehicle routing problems,” *Networks*, vol. 30, no. 2, pp. 105–119, 1997.
- [12] S. R. Thangiah and S. Salhi, “Genetic clustering for the multi-depot vehicle routing problem,” *Computers & Operations Research*, vol. 28, no. 10, pp. 1143–1151, 2001.
- [13] S. C. Ho and D. Haugland, “A tabu search algorithm for the multi-depot vehicle routing problem with time windows,” *Computers & Operations Research*, vol. 35, no. 9, pp. 2892–2908, 2008.
- [14] D. Pisinger and S. Ropke, “A general heuristic for vehicle routing problems,” *Computers & Operations Research*, vol. 34, no. 8, pp. 2403–2435, 2007.
- [15] A. Rahimi-Vahed, T. G. Crainic, and M. Gendreau, “Fleet-sizing for multi-depot and periodic vehicle routing problems using a modular heuristic algorithm,” *Computers & Operations Research*, 2015.
- [16] A. S. Azad, M. Islam, and S. Chakraborty, “A heuristic initialized stochastic memetic algorithm for mdvrp with interdependent depot operations,” *IEEE Transactions on Systems, Man, and Cybernetics*, 2017.
- [17] S. Voigt, M. Frank, P. Fontaine, and H. Kuhn, “Hybrid adaptive large neighborhood search for vehicle routing problems with depot location decisions,” *Computers & Operations Research*, vol. 140, 2022.
- [18] L. Calvet, A. Ferrer, M. I. Gomes, and A. A. Juan, “Combining statistical learning with metaheuristics for the multi-depot vehicle routing problem with market segmentation,” *Computers & Industrial Engineering*, vol. 94, pp. 93–104, 2016.
- [19] R. Shababazian, L. D. P. Pugliese, and F. Guerriero, “Integrating machine learning into vehicle routing problem: Methods and applications,” *IEEE Transactions on Intelligent Transportation Systems*, 2024.
- [20] S. Nasehi, F. Choudhury, and E. Tanin, “Deepmdv: Learning global matching for multi-depot vehicle routing problems,” *arXiv preprint arXiv:2411.17080*, 2024.
- [21] H. Xu, R. Hu, B. Qian, Z. Zhang, Q. Shang, and H. Jin, “A deep reinforcement learning method for solving the multi-depot vehicle routing problem,” *Neural Computing and Applications*, 2023.
- [22] A. Arishi and K. Krishnan, “A multi-agent deep reinforcement learning approach for solving the multi-depot vehicle routing problem,” *Journal of Management Analytics*, 2023.
- [23] E. Talbi, “Metaheuristics: From design to implementation,” *John Wiley & Sons google schola*, vol. 2, pp. 268–308, 2009.
- [24] A. A. Juan, J. Faulín, J. Jorba, D. Riera, D. Masip, and B. Barrios, “On the use of monte carlo simulation, cache and splitting techniques to improve the clark and wright savings heuristics,” *Journal of the Operational Research Society*, vol. 62, no. 6, pp. 1085–1097, 2011.
- [25] M. Peyman, X. A. Martin, J. Panadero, and A. A. Juan, “A sim-learnheuristic for the team orienteering problem: Applications to unmanned aerial vehicles,” *Algorithms*, vol. 17, no. 5, p. 200, 2024.